
🛠️ Practice: C4 Model

Think of the C4 model like Google Maps for software. We don't look at street views when we want to see a map of the continent. C4 gives us four distinct zoom levels: **Context (Level 1)**, **Container (Level 2)**, **Component (Level 3)**, and **Code (Level 4)**. For this practice, we will focus on Levels 1, 2, and 3, which you will use most in your engineering careers.

This practice walks you from **Beginner** to **Advanced**with complete step-by-step solutions and clean architectural blueprints using Mermaid syntax.

📋 Practice 1: "ByteBites" Food Delivery App

The Scenario

ByteBites is a modern food delivery startup. Customers use a mobile app to browse menus, place orders, and pay. Restaurants use a web dashboard to accept orders and update status. Delivery riders use a mobile app to navigate and accept delivery jobs. The system must integrate with an external payment gateway (Stripe) and a third-party SMS/Notification Gateway (Twilio).

🟢 Beginner Level: System Context Diagram (Level 1)

Goal: Show the big picture. Identify the core system, the human users (actors), and the external dependencies.

Step-by-Step Task Instructions

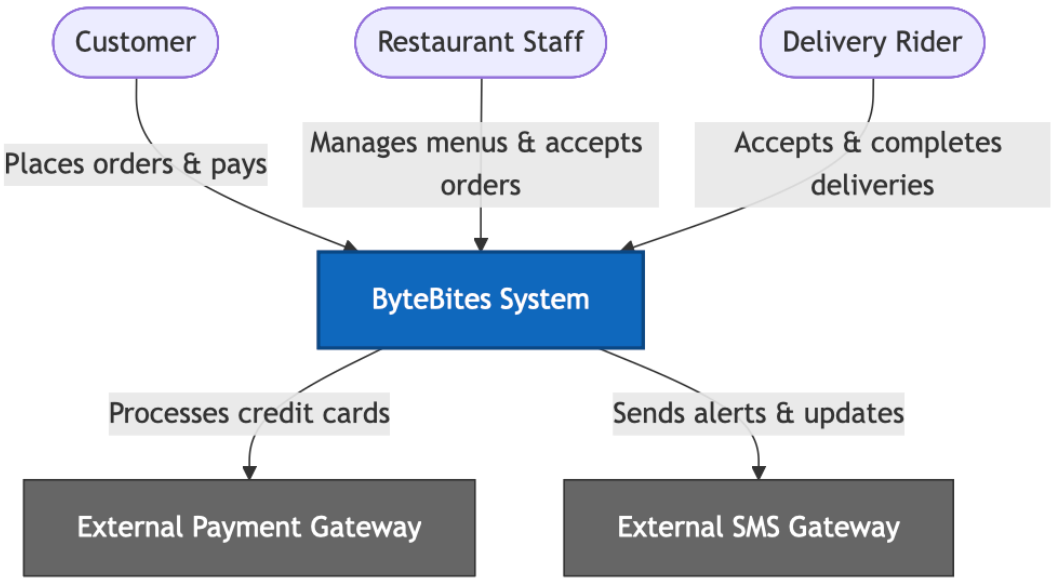
1. Identify all human actors interacting with the system.
2. Identify external software or cloud systems that ByteBites doesn't control but relies on.
3. Draw ByteBites as a single, central system block. Do not show internal technical details yet.
4. Draw directed arrows showing how data flows between actors, systems, and ByteBites.

Solution & Diagram

- **Actors:** Customer, Restaurant Staff, Delivery Rider.
- **External Systems:** Payment Gateway, SMS Gateway.
- **Core System:** ByteBites Software System.

Code snippet

```
flowchart TB
    %% Actors
    Customer([Customer]) -->|Places orders & pays| BB[ByteBites System]
    RestStaff([Restaurant Staff]) -->|Manages menus & accepts orders| BB
    Rider([Delivery Rider]) -->|Accepts & completes deliveries| BB

    %% External Systems
    BB -->|Processes credit cards| Stripe[External Payment Gateway]
    BB -->|Sends alerts & updates| Twilio[External SMS Gateway]

    style BB fill:#1168bd,stroke:#0b4884,color:#fff,stroke-width:2px
    style Stripe fill:#666,stroke:#333,color:#fff
    style Twilio fill:#666,stroke:#333,color:#fff
```



🟡 Intermediate Level: Container Diagram (Level 2)

Goal: Zoom into the ByteBites system. Show the high-level technology choices (apps, databases, backend services) and how they communicate.

Step-by-Step Task Instructions

1. Break the central "ByteBites System" box into separate deployable containers (e.g., frontend apps, backend APIs, data stores).
2. Specify the technology stack for each container (e.g., React Native, Node.js, PostgreSQL).
3. Map how the containers talk to each other (usually via JSON/HTTPS or DB drivers) and how they connect to the external systems identified in Level 1.

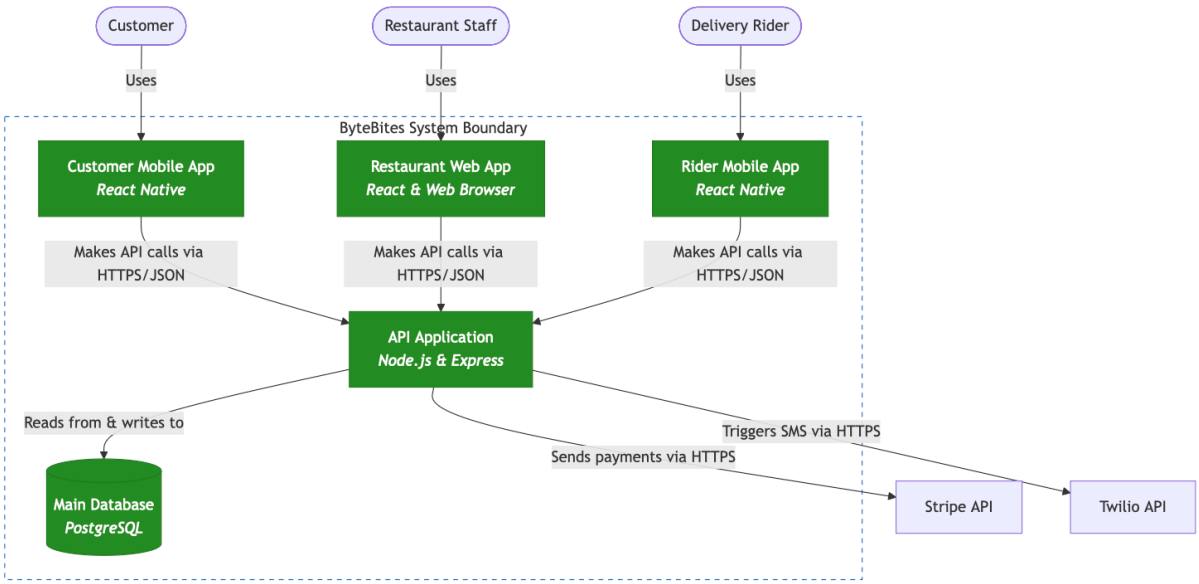
Solution & Diagram

- **Containers:** Customer Mobile App (React Native), Restaurant Web Dashboard (React), Rider Mobile App (React Native), API Application (Node.js/Express), Main Database (PostgreSQL).

Code snippet

```
flowchart TB
    subgraph ByteBites [ByteBites System Boundary]
        API[API Application<br/><i>Node.js & Express</i>]
        DB[(Main Database<br/><i>PostgreSQL</i>)]

        CApp[Customer Mobile App<br/><i>React Native</i>]
        RWeb[Restaurant Web App<br/><i>React & Web Browser</i>]
        DApp[Rider Mobile App<br/><i>React Native</i>]
    end

    %% Human interactions to containers
    Customer([Customer]) -->|Uses| CApp
    RestStaff([Restaurant Staff]) -->|Uses| RWeb
    Rider([Delivery Rider]) -->|Uses| DApp

    %% Container to container interactions
    CApp -->|Makes API calls via HTTPS/JSON| API
    RWeb -->|Makes API calls via HTTPS/JSON| API
    DApp -->|Makes API calls via HTTPS/JSON| API
```

```
API -->|Reads from & writes to| DB

%% External system interactions
API -->|Sends payments via HTTPS| Stripe[Stripe API]
API -->|Triggers SMS via HTTPS| Twilio[Twilio API]

style ByteBites fill:none,stroke:#1168bd,stroke-dasharray: 5 5
style API fill:#228b22,stroke:#1e7b1e,color:#fff
style DB fill:#228b22,stroke:#1e7b1e,color:#fff
style CApp fill:#228b22,stroke:#1e7b1e,color:#fff
style RWeb fill:#228b22,stroke:#1e7b1e,color:#fff
style DApp fill:#228b22,stroke:#1e7b1e,color:#fff
```



🔴 Advanced Level: Component Diagram (Level 3)

📱 Component Diagram: API Application (Node.js)

Goal: Zoom into a single container. Let's decompose the API Application (Node.js) backend container to understand its structural logical blocks.

Step-by-Step Task Instructions

1. Isolate the "API Application" container.
2. Identify its internal architectural modules or structural components (e.g., Controllers, Services, Repositories/DAOs).
3. Map internal dependencies: Which controller relies on which service, and which service interacts with the Database or external APIs?

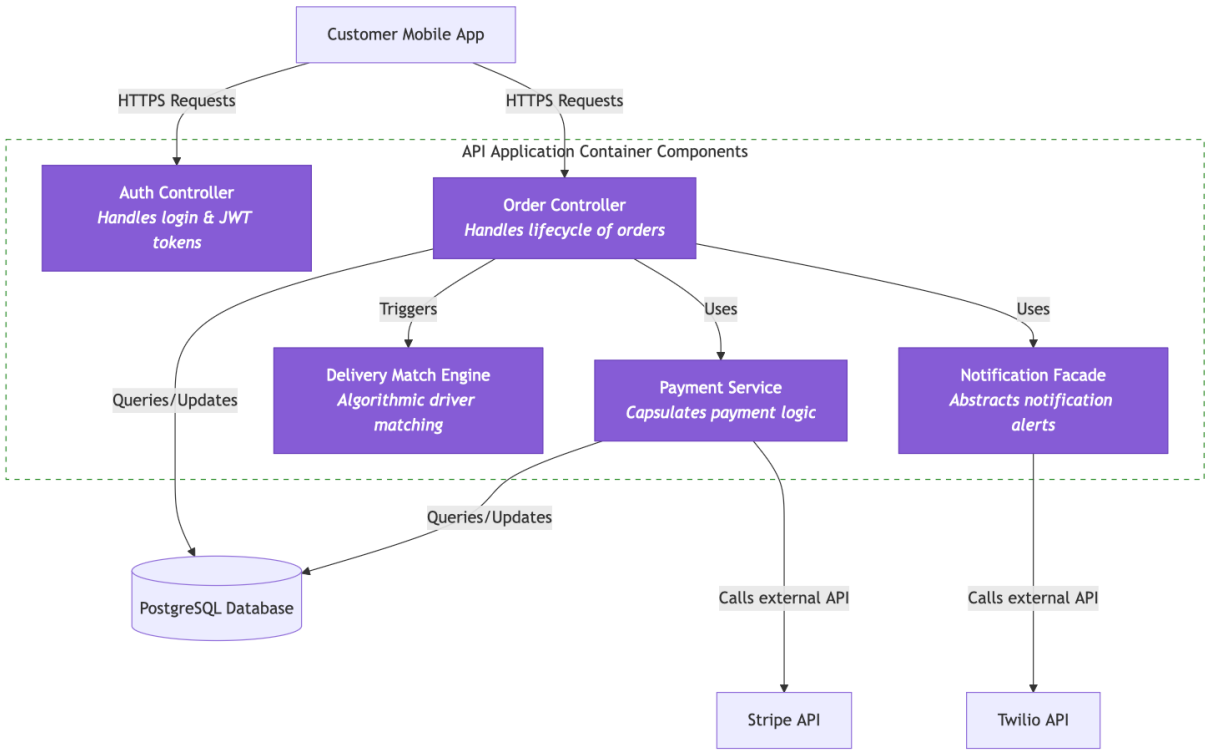
Solution & Diagram

- **Components:** Auth Controller, Order Controller, Payment Service, Delivery Matching Engine, Notification Facade.

Code snippet

```
flowchart TB
    CApp[Customer Mobile App] -->|HTTPS Requests| AuthCtrl[Auth Controller<br/><i>Handles login & JWT tokens</i>]
    CApp -->|HTTPS Requests| OrderCtrl[Order Controller<br/><i>Handles lifecycle of orders</i>]

    subgraph API [API Application Container Components]
        AuthCtrl
        OrderCtrl

        PayServ[Payment Service<br/><i>Capsulates payment logic</i>]
        MatchEng[Delivery Match Engine<br/><i>Algorithmic driver matching</i>]
        NotifFac[Notification Facade<br/><i>Abstracts notification alerts</i>]
    end

    DB[(PostgreSQL Database)]

    %% Component Wiring
```

```
OrderCtrl -->|Uses| PayServ
OrderCtrl -->|Triggers| MatchEng
OrderCtrl -->|Uses| NotifFac

PayServ -->|Queries/Updates| DB
OrderCtrl -->|Queries/Updates| DB

%% Connections extending outside container boundaries
PayServ --->|Calls external API| Stripe[Stripe API]
NotifFac --->|Calls external API| Twilio[Twilio API]

style API fill:none,stroke:#228b22,stroke-dasharray: 5 5
style AuthCtrl fill:#855cd6,stroke:#6b40bc,color:#fff
style OrderCtrl fill:#855cd6,stroke:#6b40bc,color:#fff
style PayServ fill:#855cd6,stroke:#6b40bc,color:#fff
style MatchEng fill:#855cd6,stroke:#6b40bc,color:#fff
style NotifFac fill:#855cd6,stroke:#6b40bc,color:#fff
```



📱 Component Diagram: Customer Mobile App (React Native)

Previously, we zoomed into the backend container of our **ByteBites** architecture. However, frontend apps are not just monolithic blocks of UI; they are composed of structural logical modules that manage state, route traffic, handle local storage, and interface with device hardware.

Let's expand our Advanced Level (Level 3) architectural blueprints by decomposing the remaining three frontend containers: the **Customer Mobile App**, the **Restaurant Web App**, and the **Rider Mobile App**.

Step-by-Step Task Instructions

1. **Isolate the Container:** Focus purely on the logical architecture running inside the user's smartphone.
2. **Identify Core Modules:** Break the client application down into its core structural concerns: user interface layers, local client state management, network layers, and native device hardware bridges.
3. **Map Dependencies:** Draw how UI screens query the local state manager, how the state manager delegates heavy lifting to network clients, and how specific views require local phone hardware permissions (like GPS).

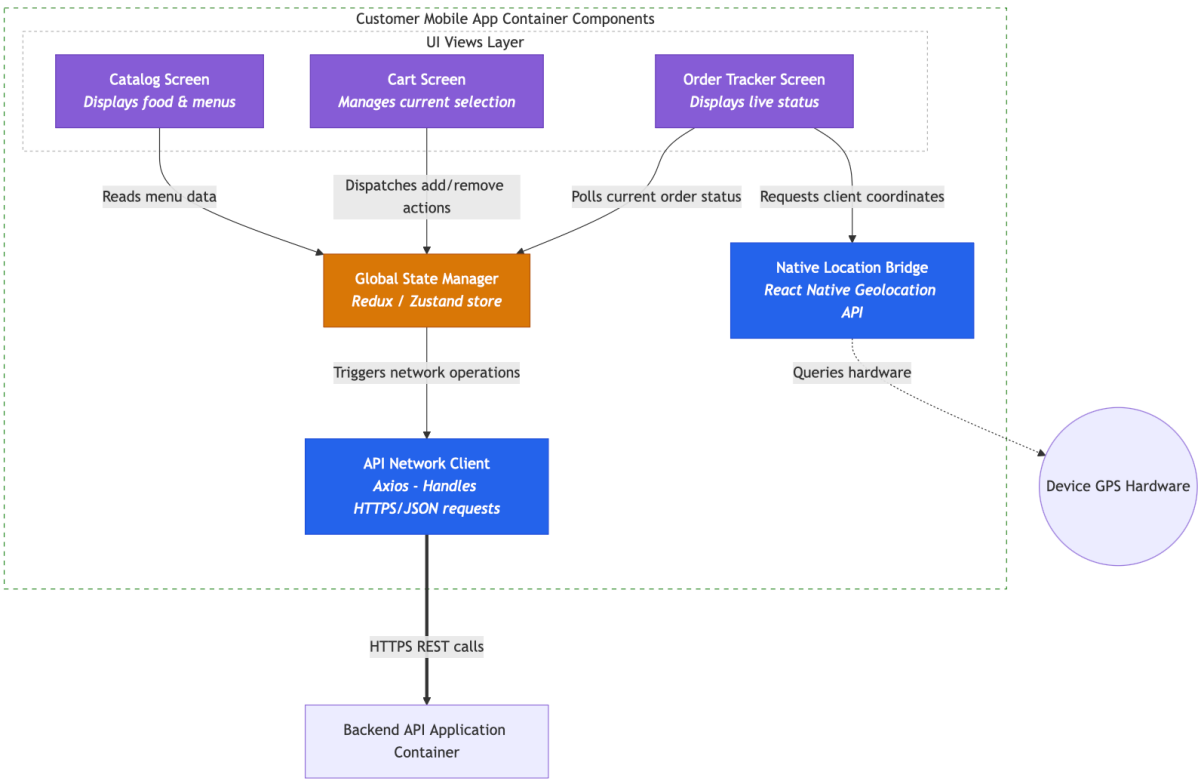
Solution & Diagram

- **Components:** UI Screens (Catalog, Cart, Order Tracker), State Manager (Redux/Zustand), API Client (Axios), Native Location Bridge (GPS).

Code snippet

```
flowchart TB
    %% External Boundary Communication
    API_App[Backend API Application Container]

    subgraph ClientApp [Customer Mobile App Container Components]
        %% UI Layer
        subgraph UI [UI Views Layer]
            Catalog[Catalog Screen<br/><i>Displays food & menus</i>]
            Cart[Cart Screen<br/><i>Manages current selection</i>]
            Tracker[Order Tracker Screen<br/><i>Displays live status</i>]
        end

        %% State & Core Logic Layer
        StateMgr[Global State Manager<br/><i>Redux / Zustand store</i>]

        %% Service Layer
        HTTPClient[API Network Client<br/><i>Axios - Handles HTTPS/JSON requests</i>]
        GPSBridge[Native Location Bridge<br/><i>React Native Geolocation API</i>]
```

```
end

%% Wiring UI to State
Catalog -->|Reads menu data| StateMgr
Cart -->|Dispatches add/remove actions| StateMgr
Tracker -->|Polls current order status| StateMgr

%% Wiring State to Services
StateMgr -->|Triggers network operations| HTTPClient
Tracker -->|Requests client coordinates| GPSBridge

%% Outbound connections
HTTPClient ===>|HTTPS REST calls| API_App
GPSBridge -.->|Queries hardware| PhoneGPS((Device GPS Hardware))

style ClientApp fill:none,stroke:#228b22,stroke-dasharray: 5 5
style UI fill:none,stroke:#b5b5b5,stroke-dasharray: 3 3
style Catalog fill:#855cd6,stroke:#6b40bc,color:#fff
style Cart fill:#855cd6,stroke:#6b40bc,color:#fff
style Tracker fill:#855cd6,stroke:#6b40bc,color:#fff
style StateMgr fill:#d97706,stroke:#b45309,color:#fff
style HTTPClient fill:#2563eb,stroke:#1d4ed8,color:#fff
style GPSBridge fill:#2563eb,stroke:#1d4ed8,color:#fff
```



💻 Component Diagram: Restaurant Web App (React & Browser)

Step-by-Step Task Instructions

1. **Isolate the Container:** Focus on the modular structure running entirely within a desktop web browser engine.
2. **Identify Real-Time Constraints:** Restaurants need split-second updates when an order is placed. Identify the module responsible for open event-driven connections (WebSockets) alongside the standard HTTP transaction module.
3. **Map Internal Logic:** Show how incoming event notifications change the global view state without requiring a full browser page refresh.

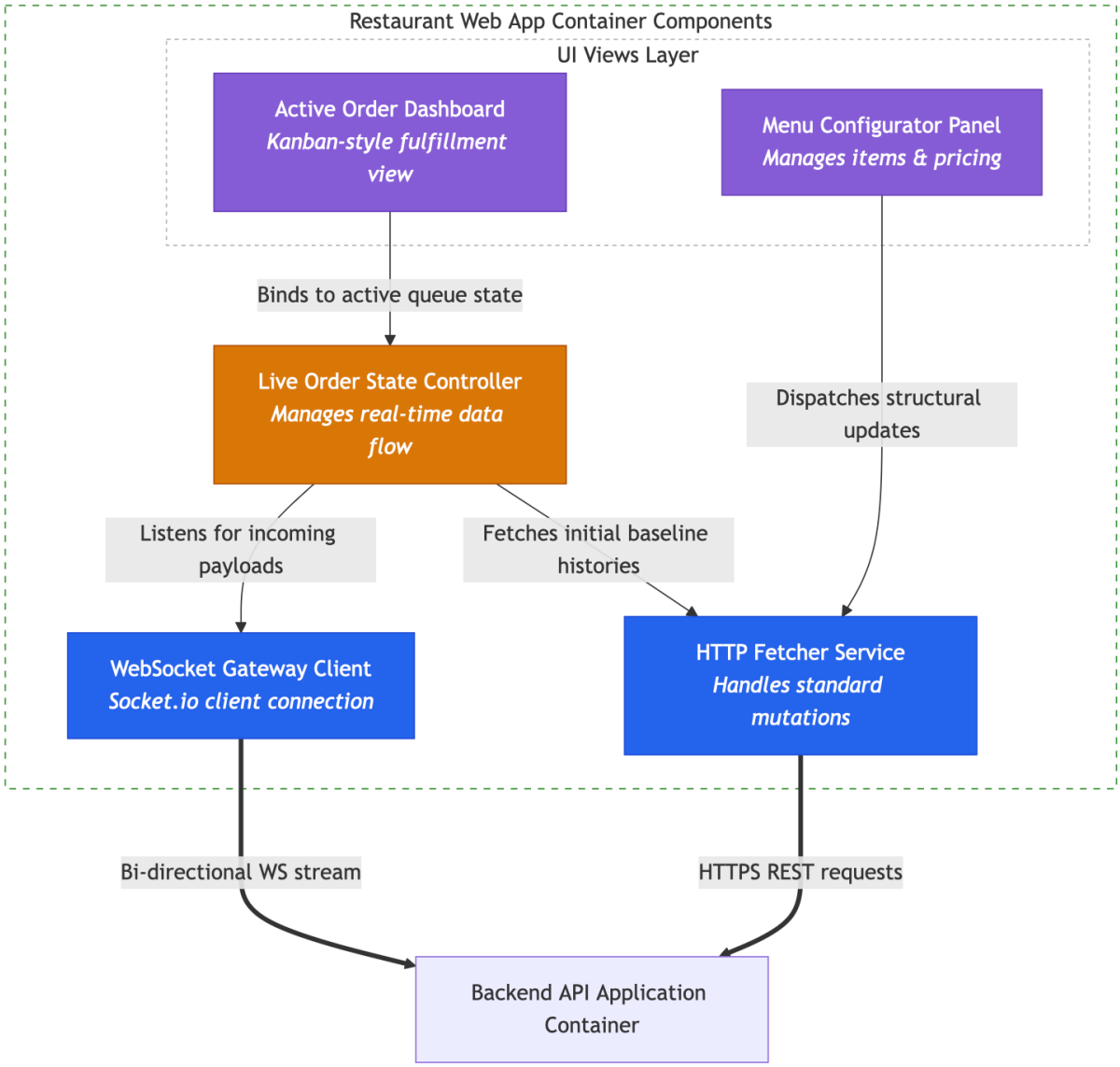
Solution & Diagram

- **Components:** UI Layouts (Order Dashboard, Menu Configurator), Live Order Sync Manager, WebSocket Client, HTTP Data Fetcher.

Code snippet

```
flowchart TB
    %% External Boundary Communication
    API_App[Backend API Application Container]

    subgraph WebApp [Restaurant Web App Container Components]
        subgraph WebUI [UI Views Layer]
            Dashboard[Active Order Dashboard<br/><i>Kanban-style fulfillment view</i>]
            MenuEditor[Menu Configurator Panel<br/><i>Manages items & pricing</i>]
        end

        %% Real-time and application state controllers
        SyncMgr[Live Order State Controller<br/><i>Manages real-time data flow</i>]

        %% Infrastructure Services
        WSClient[WebSocket Gateway Client<br/><i>Socket.io client connection</i>]
        AxiosClient[HTTP Fetcher Service<br/><i>Handles standard mutations</i>]
    end

    %% Wiring views to logic controllers
    Dashboard -->|Binds to active queue state| SyncMgr
    MenuEditor -->|Dispatches structural updates| AxiosClient

    %% Wiring sync controllers to network clients
    SyncMgr -->|Listens for incoming payloads| WSClient
    SyncMgr -->|Fetches initial baseline histories| AxiosClient

    %% Outbound network boundaries
    WSClient ===>|Bi-directional WS stream| API_App
    AxiosClient ===>|HTTPS REST requests| API_App
```

```
style WebApp fill:none,stroke:#228b22,stroke-dasharray: 5 5
style WebUI fill:none,stroke:#b5b5b5,stroke-dasharray: 3 3
style Dashboard fill:#855cd6,stroke:#6b40bc,color:#fff
style MenuEditor fill:#855cd6,stroke:#6b40bc,color:#fff
style SyncMgr fill:#d97706,stroke:#b45309,color:#fff
style WSClient fill:#2563eb,stroke:#1d4ed8,color:#fff
style AxiosClient fill:#2563eb,stroke:#1d4ed8,color:#fff
```



🛵 Component Diagram: Rider Mobile App (React Native)

Step-by-Step Task Instructions

1. **Isolate the Container:** Focus on the specialized application context built for delivery riders on the move.
2. **Address Critical Background Tasks:** Unlike regular apps, a delivery tracker app must continuously broadcast location telemetries *even when the phone is locked or in the rider's pocket*. Isolate this background worker module.
3. **Draw Component Interactions:** Document how incoming shift offers hit the view layer and how map components utilize hardware tracking coordinates to overlay optimal navigation routing path lines.

Solution & Diagram

- **Components:** UI Screens (Job Board, Active Navigation Map), Tracking Daemon Service, API Client, Native Map Renderer SDK.

Code snippet

```
flowchart TB
    %% External Boundary Communication
    API_App[Backend API Application Container]

    subgraph RiderApp [Rider Mobile App Container Components]
        subgraph RiderUI [UI Views Layer]
            JobBoard[Available Job Board<br/><i>Accepts or rejects delivery offers</i>]
            NavMap[Active Delivery Map Screen<br/><i>Renders route lines to customer</i>]
        end

        %% Core operational background controllers
        Daemon[Background Tracking Worker<br/><i>Ensures continuous execution</i>]

        %% Specialized Service wrappers
        MapSDK[Native Map SDK Handler<br/><i>Google Maps / Apple Maps bridge</i>]
        RiderNetwork[API Gateway Sync Client<br/><i>Handles dispatch data frames</i>]
    end

    %% Wiring Views to Controllers and Wrappers
    JobBoard -->|Triggers accept action status| RiderNetwork
    NavMap -->|Queries visual route matrix layers| MapSDK
    NavMap -->|Subscribes to spatial changes| Daemon

    %% Controller actions to networks
    Daemon -->|Pushes high-priority current coordinates| RiderNetwork

    %% Outbound system communication frames
```

```
RiderNetwork ===>|HTTPS POST coordinate chunks| API_App
MapSDK -.->|Streams map graphic rendering| MapService((External Map Tiles API))
Daemon -.->|Polled coordinates| HardwareGPS((Device GPS Module))

style RiderApp fill:none,stroke:#228b22,stroke-dasharray: 5 5
style RiderUI fill:none,stroke:#b5b5b5,stroke-dasharray: 3 3
style JobBoard fill:#855cd6,stroke:#6b40bc,color:#fff
style NavMap fill:#855cd6,stroke:#6b40bc,color:#fff
style Daemon fill:#d97706,stroke:#b45309,color:#fff
style MapSDK fill:#2563eb,stroke:#1d4ed8,color:#fff
style RiderNetwork fill:#2563eb,stroke:#1d4ed8,color:#fff
```